ZapShift

Parcel Delivery Application

Project Documentation

Full-Stack Web Application

Developed by Md.Mesbahul Alam

https://attractive-optimism-production.up.railway.app/

https://github.com/mesbahtoha/Percel-Web-Application

2026

Table of Contents

1.	Project Overview	3
2.	System Architecture	4
3.	User Roles & Features	5
4.	Delivery Workflow	6
5.	Tech Stack & Dependencies	7
6.	Database Schema	9
7.	API Endpoints	10
8.	Security Implementation	11
9.	Payment Integration	12
10.	Setup & Installation Guide	13

1. Project Overview

ZapShift is a modern full-stack parcel delivery management system designed to streamline the logistics experience for three distinct user groups: Customers, Riders, and Administrators. The platform eliminates manual coordination by providing real-time tracking, automated notifications, and a secure payment gateway.

1.1 Vision & Goals

- Provide a seamless parcel booking experience for end users
- Empower delivery riders with a dedicated mobile-friendly interface
- Give administrators full operational control and live analytics
- Ensure secure, scalable infrastructure for growing delivery demand

1.2 Core Highlights

Category	Detail
Platform	Web Application (React SPA + Node.js REST API)
User Roles	Customer | Rider | Admin
Auth	Firebase Authentication + JWT Token Authorization
Payments	Stripe Payment Gateway (card processing)
Database	MongoDB Atlas (NoSQL, cloud-hosted)
Maps	React Leaflet — interactive parcel tracking map
Notifications	Nodemailer for email delivery confirmations
Deployment	Netlify (frontend) | Railway / Render (backend)

2. System Architecture

ZapShift follows a classic three-tier client-server architecture with a React SPA on the frontend, Express.js REST API on the backend, and MongoDB as the persistence layer. Firebase handles client-side authentication, while JWT tokens secure all API routes.

2.1 Architecture Diagram (Text)

```
[ Browser / Mobile Browser ]
React SPA  (Vite + Tailwind + DaisyUI)
↕  HTTPS / REST API  ( Axios + TanStack Query )
Express.js REST API  ( Node.js )
JWT Middleware  |  Firebase Admin  |  Stripe SDK  |  Nodemailer
↕  Mongoose / Native Driver
MongoDB Atlas  ( Cloud NoSQL Database )
```

2.2 Key Design Decisions

- TanStack Query replaces useEffect for all data fetching — automatic caching, background refetch, and loading/error states out of the box
- JWT + Firebase dual-layer security — Firebase handles sign-in, JWT protects API routes
- MongoDB chosen for flexible schema — parcel objects vary significantly by type and status
- React Context + TanStack for state management — no Redux overhead needed

3. User Roles & Features

3.1 Customer (User)

#	Feature	Description
1	**User Registration & Login**	Firebase auth (email/password, Google OAuth)
2	**Book a Parcel**	Form with sender/receiver info, weight, type, region
3	**Payment**	Stripe card payment per booking
4	**Parcel History**	Full list of past parcels with status & cost
5	**Live Tracking**	React Leaflet map showing current parcel location
6	**Email Notifications**	Nodemailer sends status-change emails automatically
7	**Reviews & Ratings**	Submit star ratings & feedback after delivery
8	**Profile Management**	Update personal info and view account summary

3.2 Rider

#	Feature	Description
1	**Assigned Parcel Queue**	View all parcels assigned by admin with full details
2	**Status Updates**	Mark parcel: Picked Up → In Transit → Delivered
3	**OTP Delivery Confirmation**	Secure 6-digit OTP to verify handoff to recipient
4	**Earnings Summary**	Daily breakdown of deliveries and earnings
5	**Location Sharing**	Live GPS coordinates sent for customer tracking

3.3 Administrator

#	Feature	Description
1	**Parcel Oversight**	View, filter & manage all parcels across the platform
2	**Rider Assignment**	Manually assign parcels to available riders by region
3	**User Management**	Create, suspend, or delete user/rider accounts
4	**Analytics Dashboard**	Charts for daily deliveries, revenue & performance KPIs

5	**Warehouse Management**	Manage regional hubs for inter-district parcels
6	**Manual Status Override**	Update any parcel status or reassign riders
7	**Revenue Reports**	Filter earnings by date range, region, or rider

4. Delivery Workflow

Every parcel follows a defined lifecycle managed by the three roles. Below is the complete end-to-end flow from booking to delivery confirmation.

4.1 Standard Delivery Flow

Step	Status	Action & Actor
1	**Booked (Unpaid)**	Customer fills the booking form and submits. Parcel created with 'Unpaid' status.
2	**Paid**	Customer completes Stripe payment. Status updates to 'Paid'.
3	**Ready to Pickup**	Admin reviews, assigns a rider for collection. Status becomes 'Ready to Pickup'.
4	**In Transit**	Rider physically collects the parcel and marks it 'Picked Up' → 'In Transit'.
5a	**Delivered ✓**	Same-city delivery: Rider delivers, recipient confirms via OTP. Done.
5b	**At Warehouse**	Out-of-city: Rider drops at regional warehouse. Status: 'At Warehouse'.
6b	**Shipped**	Warehouse staff ships parcel to destination hub. Status: 'Shipped'.
7b	**Delivered ✓**	Destination rider collects, delivers, OTP confirmed. Status: 'Delivered'.

4.2 Pricing Structure

Type	Weight	Within City	Outside City
Document	Any	**৳60**	**৳80**
Non-Document	Up to 3 kg	**৳110**	**৳150**
Non-Document	Above 3 kg	৳ 110 + ৳40/kg extra	৳ 150 + ৳40/kg + ৳40

5. Tech Stack & Dependencies

5.1 Frontend Packages

Package	Version	Purpose	Category
React	18.x	Core UI library for building the single-page application	Core
React Router DOM	6.x	Client-side routing and navigation management	Core
Tailwind CSS	3.x	Utility-first CSS framework for rapid UI styling	Styling
DaisyUI	4.x	Pre-built Tailwind component library (modals, cards, buttons)	Styling
Lucide React	latest	Modern clean icon set used throughout the UI	UI
React Icons	latest	Additional icon library for extended icon coverage	UI
Urbanist Font	Google	Custom sans-serif typography imported via index.css	UI
React Responsive Carousel	latest	Homepage banner/hero image slider component	UI
React Fast Marquee	latest	Smooth horizontal scrolling announcement banners	UI
React AOS	latest	Animate On Scroll — entrance animations on page scroll	Animation
React Hook Form	latest	Performant form state management and validation	Forms
React Select	latest	Advanced searchable dropdown / multi-select component	Forms
React Leaflet	latest	Interactive map wrapper for the Leaflet.js library	Maps
MapContainer / TileLayer / Marker / Popup	Leaflet	Core map primitives for parcel tracking visualization	Maps
SweetAlert2	latest	Beautiful accessible alert and confirmation dialogs	UX
sweetalert2-react-content	latest	React JSX content integration for SweetAlert2 modals	UX
Firebase (client)	latest	Client-side auth (email, Google), cloud services	Auth
@stripe/react-stripe-js	latest	Stripe payment element and card UI components	Payment

@tanstack/react-query	latest	Server state management, caching, background refetch — replaces useEffect for API calls	Data
Axios	latest	Promise-based HTTP client for REST API communication	Data
UUID	latest	Generates unique identifiers for parcel tracking numbers	Utility

5.2 Backend Packages

Package	Version	Purpose	Category
Node.js	16+	JavaScript runtime environment powering the server	Core
Express.js	4.x	Fast, minimal web framework for building the REST API	Core
MongoDB (Atlas)	latest	NoSQL cloud database storing users, parcels, orders	Database
dotenv	latest	Loads environment variables from .env into process.env	Config
cors	latest	Cross-origin resource sharing middleware for API access	Middleware
JWT (jsonwebtoken)	latest	Signs and verifies tokens for protected route authorization	Security
firebase-admin	latest	Server-side Firebase SDK for token verification	Security
Nodemailer	latest	Sends transactional emails (booking confirmations, updates)	Notifications
Stripe (server SDK)	latest	Processes payments securely on the backend	Payment

6. Database Schema

6.1 Users Collection

```
{
  "_id"        : ObjectId,
  "name"       : String,
  "email"      : String,         // unique index
  "photoURL"   : String,
  "role"       : "user" | "rider" | "admin",
  "phone"      : String,
  "address"    : String,
  "region"     : String,         // for riders
  "createdAt"  : Date
}
```

6.2 Parcels Collection

```
{
  "_id"            : ObjectId,
  "trackingId"     : String,     // UUID-based unique ID
  "senderId"       : String,     // Firebase UID
  "senderName"     : String,
  "senderPhone"    : String,
  "senderAddress"  : String,
  "receiverName"   : String,
  "receiverPhone"  : String,
  "receiverAddress": String,
  "receiverRegion" : String,
  "parcelType"     : "document" | "non-document",
  "weight"         : Number,     // kg
  "cost"           : Number,     // BDT
  "status"         : "unpaid" | "paid" | "ready_to_pickup" | "in_transit" |
"delivered",
  "riderId"        : String,     // assigned rider UID
  "paymentId"      : String,     // Stripe payment intent ID
  "latitude"       : Number,     // current location
  "longitude"      : Number,
  "createdAt"      : Date,
  "updatedAt"      : Date
}
```

6.3 Reviews Collection

```
{
  "_id"       : ObjectId,
  "parcelId"  : String,
```

```
  "userId"    : String,
  "rating"    : Number,   // 1-5
  "comment"   : String,
  "createdAt" : Date
}
```

7. API Endpoints

All endpoints are prefixed with the base URL. JWT-protected routes require an Authorization: Bearer <token> header. Admin routes additionally check the user's role claim in the database.

Method	Endpoint	Auth	Description
POST	/api/auth/verify-token	Public	Verify Firebase ID token, issue JWT
GET	/api/users/me	JWT	Get current authenticated user profile
PATCH	/api/users/:id/role	Admin	Update user role (promote to rider/admin)
GET	/api/parcels	Admin	List all parcels with filters (status, region, date)
POST	/api/parcels	JWT	Create new parcel booking
GET	/api/parcels/my	JWT	Get parcels belonging to authenticated user
GET	/api/parcels/:id	JWT	Get single parcel details by ID
PATCH	/api/parcels/:id/status	JWT	Update parcel status (rider or admin only)
PATCH	/api/parcels/:id/assign	Admin	Assign a rider to a parcel
POST	/api/payments/intent	JWT	Create Stripe payment intent for a parcel
POST	/api/payments/confirm	JWT	Confirm payment and update parcel to 'Paid'
GET	/api/riders/available	Admin	List riders available in a given region
POST	/api/reviews	JWT	Submit a review for a completed delivery
GET	/api/reviews/:parcelId	Public	Get reviews for a specific parcel
GET	/api/admin/stats	Admin	Aggregate delivery counts, revenue, KPIs

8. Security Implementation

8.1 Authentication Flow

ZapShift uses a two-layer authentication strategy: Firebase for client-side identity and JWT for server-side authorization.

#	Layer	Implementation Detail
1	**Firebase Auth**	User logs in via email/password or Google OAuth. Firebase returns an ID token.
2	**Token Exchange**	Client sends Firebase ID token to /api/auth/verify-token. Server verifies with firebase-admin SDK.
3	**JWT Issuance**	Server signs a short-lived JWT (HS256) containing user ID, email, and role. Sent back to client.
4	**API Authorization**	Every protected request includes Authorization: Bearer <jwt>. Middleware verifies signature and expiry.
5	**Role Check**	Admin-only routes check req.user.role === 'admin' after JWT verification.

8.2 Security Measures

- Environment variables (.env) — all secrets kept out of source code
- CORS whitelist — only the deployed frontend domain is allowed
- RBAC (Role-Based Access Control) — User, Rider, Admin each see/access only their permitted resources
- OTP delivery confirmation — prevents parcel interception or false delivery claims
- Stripe client-server separation — card data never touches our server; processed via Stripe SDK
- Firebase Admin SDK verification — prevents token forgery on the backend

9. Payment Integration

ZapShift uses Stripe's Payment Intents API for secure parcel payment processing. The flow follows Stripe's recommended server-side intent creation pattern.

9.1 Payment Flow

Step	Action
1	User clicks 'Pay Now' on a booked parcel page
2	Client calls POST /api/payments/intent — server creates a Stripe PaymentIntent and returns clientSecret
3	React Stripe.js CardElement collects card details securely in Stripe's hosted iframe (PCI compliant)
4	stripe.confirmCardPayment(clientSecret) is called client-side with card data
5	On success, client calls POST /api/payments/confirm — server verifies with Stripe and updates parcel status to 'Paid'
6	Nodemailer sends a payment confirmation email to the user

9.2 Packages Used

- @stripe/react-stripe-js — CardElement, useStripe, useElements hooks for the React UI
- stripe (npm, server) — PaymentIntent creation, payment verification on backend
- Webhook support ready — server can optionally handle stripe-signature verified webhooks

10. Setup & Installation Guide

10.1 Prerequisites

- Node.js v16 or higher
- npm v8+ or yarn
- MongoDB Atlas account (free tier works)
- Firebase project with Authentication enabled
- Stripe account (test keys for development)
- Gmail account with App Password for Nodemailer

10.2 Clone & Install

```
# Clone the repository
git clone https://github.com/mesbahtoha/Zap-Shift-Parcel-Delivery-Application.git
cd Zap-Shift-Parcel-Delivery-Application

# Install frontend dependencies
cd client && npm install

# Install backend dependencies
cd ../server && npm install
```

10.3 Environment Variables

Client (.env)

```
VITE_FIREBASE_API_KEY=your_api_key
VITE_FIREBASE_AUTH_DOMAIN=your_project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your_project_id
VITE_FIREBASE_STORAGE_BUCKET=your_bucket
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_app_id
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_xxxxxxxx
VITE_API_BASE_URL=http://localhost:5000
```

Server (.env)

```
PORT=5000
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/zapshift
JWT_SECRET=your_super_secret_jwt_key
STRIPE_SECRET_KEY=sk_test_xxxxxxxx
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_gmail_app_password
FIREBASE_ADMIN_CREDENTIALS='{...firebase service account JSON...}'
```

10.4 Run the Application

```
# Terminal 1 — Start the backend server
cd server
node index.js
# Server running on http://localhost:5000

# Terminal 2 — Start the frontend dev server
cd client
npm run dev
# App running on http://localhost:5173
```

10.5 Build for Production

```
# Build the React frontend
cd client && npm run build

# Deploy /dist folder to Netlify or Vercel
# Deploy /server to Railway, Render, or any Node.js host
```

ZapShift — Delivering Fast, Delivering Smart

Developed by Mesbahul Alam | github.com/mesbahtoha | mesbahulalam017@gmail.com